H1 Report

- Name : 潘姵蓁
- ID : D1388259
- 系級 : 資訊二丁

題目：暗棋遊戲

撰寫一個簡單版、非 GUI 介面的 Chess 系統。使用者可以在 console 介面輸入所要選擇的棋子的位置 (例如 A2, B3)，若該位置的棋子未翻開則翻開，若已翻開則系統要求輸入目的的位置進行移動或吃子，如果不成功則系統提示錯誤回到原來狀態。每個動作都會重新顯示棋盤狀態。

設計方法概述

- 遊戲棋盤為 4x8 的格子，共 32 顆棋子
- 每顆棋子由 `Chess` 類別建立，包含：
  - 名稱 (車、馬、兵...)
  - 大小 (判斷吃子優先權)
  - 陣營 (紅 / 黑)
  - 位置 (A1, B3...)
  - 是否翻開
- `ChessGame` 類別管理整個遊戲，繼承自抽象類別 `AbstractGame`
- 抽象方法：
  - `setPlayers(Player p1, Player p2)`：設定玩家
  - `boolean gameOver()`：判斷遊戲是否結束
  - `boolean move(String from, String to, Player player)`：翻棋或移動吃子
  - `void generateChess()`：產生並隨機排列棋子
  - `void showAllChess()`：顯示棋盤狀態
  - `Chess getChessAt(String loc)`：取得指定位置的棋子
  - `String getWinner()`：取得勝利者

程式、執行畫面及其說明

Chess.java

```java
public class Chess {
    String name;     // 棋子名稱 ( 車、馬… )
    int weight;      // 棋子大小 ( 用來判斷吃子 )
    String side;     // 陣營 ( 紅 / 黑 )
    String loc;      // 位置 ( A1, B2… )
    boolean isOpen; // 是否翻開
    boolean isEaten; // 是否已被吃掉

    public Chess(String name, int weight, String side, String loc) {
```

```
        this.name = name;
        this.weight = weight;
        this.side = side;
        this.loc = loc;
        this.isOpen = false;
        this.isEaten = false;
    }

    public void flip() {
        isOpen = true;
    }

    @Override
    public String toString() {
        if(isEaten) return "*";  // 被吃掉的棋子
        if(!isOpen) return "◦";  // 未翻開的棋子
        return side + name;      // 已翻開顯示名稱
    }
}
```

ChessGame.java核心設計

隨機放置棋子

```
Collections.shuffle(pieces); // 將 32 顆棋子打亂順序
for(int r=0;r<ROWS;r++){
    for(int c=0;c<COLS;c++){
        Chess ch = pieces.get(r*COLS + c);
        ch.loc = "" + (char)('A'+r) + (c+1); // 分配位置 A1~D8
    }
}
```

判斷棋子大小與吃子邏輯

```
if(to.weight <= from.weight){  // 可以吃
    to.side = from.side;
    to.name = from.name;
    to.isOpen = true;
    from.isEaten = true;
    System.out.println(from.side + from.name + " 吃掉 " + to.side + to.name);
} else {
    System.out.println("無法吃掉比自己大的棋子！");
    return false;
}
```

控制玩家只能操作自己棋子

```
if(!from.side.equals(player.side)){
    System.out.println("不能操作對方棋子！");
    return false;
}
```

操作錯誤偵錯

```
-選到不存在的位置 → 提示「起始位置不存在」

-翻開已翻棋 → 提示「該棋子已翻開」

-移動到未翻開棋 → 提示「目的位置未翻開」

-嘗試吃自己棋 → 提示「不能吃自己棋子」

-操作失敗 → 回到原來狀態，不換玩家
```

顯示棋盤

```
System.out.print("   ");
for(int c=1;c<=COLS;c++) System.out.print(" " + c + " ");
System.out.println();

for(int r=0;r<ROWS;r++){
    System.out.print((char)('A'+r) + "  ");
    for(int c=0;c<COLS;c++){
        Chess ch = getChessAt("" + (char)('A'+r) + (c+1));
        System.out.print(" " + (ch==null ? "∘" : ch.toString()) + " ");
    }
    System.out.println();
}
```

執行畫面示意

```
    1  2  3  4  5  6  7  8
A   ∘  ∘  ∘  ∘  ∘  ∘  ∘  ∘
B   ∘  ∘  ∘  ∘  ∘  ∘  ∘  ∘
C   ∘  ∘  ∘  ∘  ∘  ∘  ∘  ∘
D   ∘  ∘  ∘  ∘  ∘  ∘  ∘  ∘
玩家1 的回合（陣營：尚未決定）
輸入棋子位置進行翻棋/移動：A2
輸入目的位置（翻棋請與起始位置相同）：A2
玩家1 翻開 黑將

玩家2 的回合（陣營：尚未決定）
```

```
輸入棋子位置進行翻棋/移動：B3
輸入目的位置：B3
玩家2 翻開 紅車

玩家1 的回合（陣營：黑）
輸入棋子位置進行翻棋/移動：A2
輸入目的位置：B3
黑將 吃掉 紅車
```

*註：。 表示未翻開， * 表示被吃掉，紅/黑 + 名稱 表示已翻開棋子

棋子大小表

```
棋子名稱 | 紅方權重 | 黑方權重
--------------------------------
將 / 帥   | 10       | 10
士 / 仕   | 4        | 4
象 / 相   | 3        | 3
車 / 俥   | 5        | 5
馬 / 傌   | 5        | 5
包 / 炮   | 6        | 6
兵 / 卒   | 1        | 1
```

吃子規則表

```
規則：
- 玩家只能吃自己陣營棋子權重大於等於對方的棋子
- 不能吃自己陣營棋子
- 若大小不足 → 提示「無法吃掉比自己大的棋子」
```

移動示意圖

```
棋盤示意 (A-D x 1-8)
翻棋：輸入同位置
移動 / 吃子：
   1  2  3  4  5  6  7  8
A  。  將  。  。  。  。  。  。
B  。  車  。  。  。  。  。  。
C  。  。  。  。  。  。  。  。
D  。  。  。  。  。  。  。  。
操作：黑將從 A2 移動到 B2 吃掉紅車
```

AI 使用狀況與心得

使用層級：層級2（除錯 + 改善功能與架構）

互動次數與內容：

```
1.整理 ChessGame 的翻棋/移動邏輯

2.修正第一次翻棋就結束的 bug

3.設計吃子規則與玩家操作限制

4.提供棋盤輸出與錯誤偵測提示
```

自己手動的部分：

```
1.撰寫 Chess 類別

2.Main 遊戲流程與玩家輸入

3.圖片截圖

4.PDF 轉檔
```

心得：

```
1.AI 幫我省下了大量邏輯推敲時間

2.澄清棋子翻棋、吃子和移動規則

3.自行驗證 AI 提供邏輯是否符合暗棋規則

4.深入了解 OOP 的繼承、抽象類別與物件互動
```

心得

```
1.學到迴圈與集合應用 (ArrayList、for 迴圈)

2.熟悉抽象類別 AbstractGame 與繼承的重要性

3.練習 console 輸入輸出、字串格式化

4.Git 與版本控制的重要性

5.Maven 專案可用 VSCode 編輯管理

6.AI 協助除錯與程式整理效率高，但邏輯需自行確認
```